

Atomic Design

Brad Frost

Chapter 2

Atomic Design Methodology

Atoms, molecules, organisms, templates, and pages

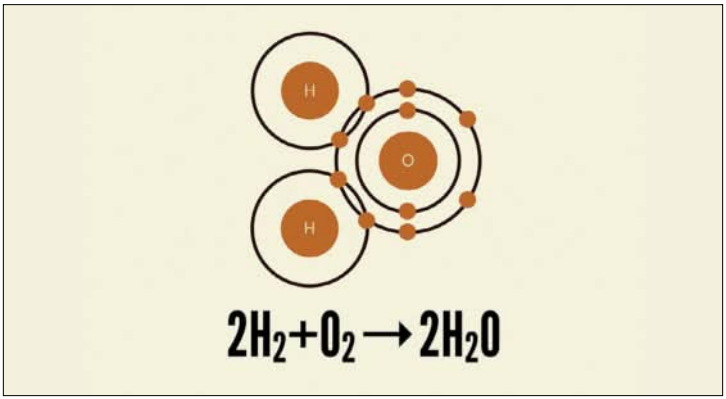
My search for a methodology to craft interface design systems led me to look for inspiration in other fields and industries. Given this amazingly complex world we've created, it seemed only natural that other fields would have tackled similar problems we could learn from and appropriate. As it turns out, loads of other fields such as industrial design and architecture have developed smart modular systems for manufacturing immensely complex objects like airplanes, ships, and skyscrapers.

But my original explorations kept creeping back to the natural world, which triggered memories of sitting at a rickety desk in my high school's chemistry lab.

Taking cues from chemistry

My high school chemistry class was taught by a no-nonsense Vietnam vet with an extraordinarily impressive mustache. Mr. Rae's class had a reputation for being one of the hardest classes in school, largely because of an assignment that required students to balance hundreds upon hundreds of chemical equations contained in a massive worksheet.

If you're like me, you may need a bit of a refresher to recall what a chemical equation looks like, so here you go:



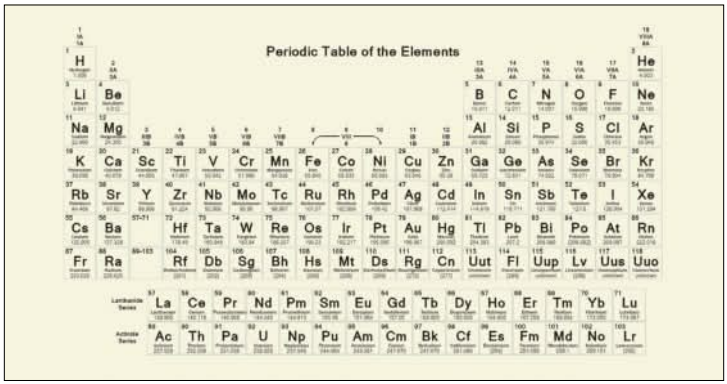
An example of a chemical equation showing hydrogen and oxygen atoms combining together to form a water molecule.

Chemical reactions are represented by chemical equations, which often show how atomic elements combine together to form molecules. In the example above, we see how hydrogen and oxygen combine together to form water molecules.

In the natural world, **atomic elements combine together to form molecules. These molecules can combine further to form relatively complex organisms.** To expound a bit further:

- **Atoms** are the basic building blocks of all matter. Each chemical element has distinct properties, and they can't be broken down further without losing their meaning. (Yes, it's true atoms are composed of even smaller bits like protons, electrons, and neutrons, but atoms are the smallest *functional* unit.)
- **Molecules** are groups of two or more atoms held together by chemical bonds. These combinations of atoms take on their own unique properties, and become more tangible and operational than atoms.
- **Organisms** are assemblies of molecules functioning together as a unit. These relatively complex structures can range from single-celled organisms all the way up to incredibly sophisticated organisms like human beings.

Of course, I'm simplifying the incredibly rich composition of the universe, but the basic gist remains: atoms combine together to form molecules, which further combine to form organisms. This atomic theory means that all matter in the known universe can be broken down into a finite set of atomic elements:



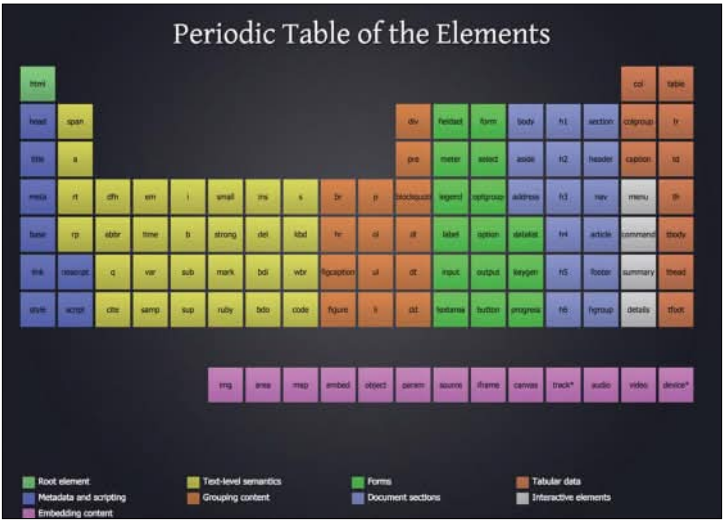
The periodic table of chemical elements.

Apparently Mr. Rae's strategy of having students mind-numbingly balance tons of chemical equations worked, because I'm coming back to it all these years later for inspiration on how to approach interface design.

The atomic design methodology

By now you may be wondering why we're talking about atomic theory, and maybe you're even a bit angry at me for forcing you to relive memories of high school chemistry class. But this is going somewhere, I promise.

We discussed earlier how all matter in the universe can be broken down into a finite set of atomic elements. As it happens, our interfaces can be broken down into a similar finite set of elements. Josh Duck's Periodic Table of HTML Elements beautifully articulates how all of our websites, apps, intranets, hoobadyboops, and whatevers are all composed of the same HTML elements.



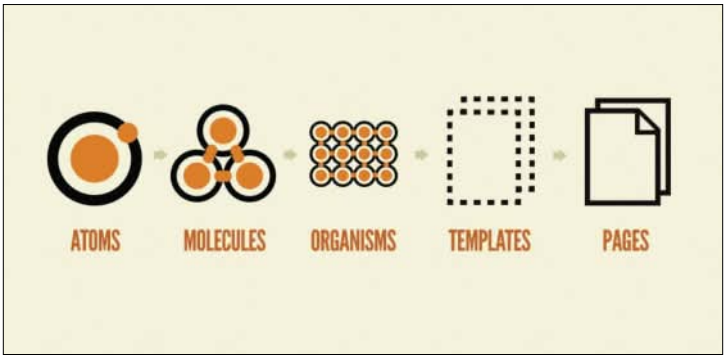
The periodic table of HTML elements by Josh Duck.

Because we're starting with a similar finite set of building blocks, we can apply that same process that happens in the natural world to design and develop our user interfaces.

Enter atomic design.

Atomic design is a methodology composed of five distinct stages working together to create interface design systems in a more deliberate and hierarchical manner. The five stages of atomic design are:

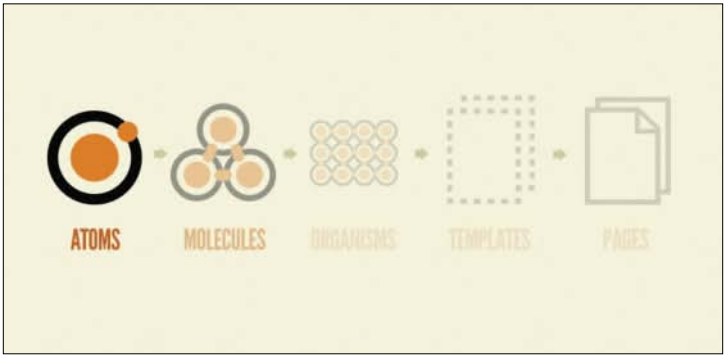
1. Atoms
2. Molecules
3. Organisms
4. Templates
5. Pages



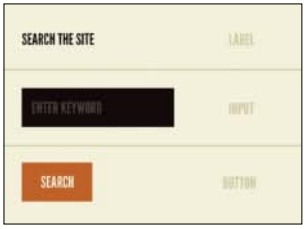
Atomic design is atoms, molecules, organisms, templates, and pages concurrently working together to create effective interface design systems.

Atomic design is not a linear process, but rather a mental model to help us think of our user interfaces as both a cohesive whole and a collection of parts *at the same time*. Each of the five stages plays a key role in the hierarchy of our interface design systems. Let's dive into each stage in a bit more detail.

Atoms



If atoms are the basic building blocks of matter, then the **atoms of our interfaces serve as the foundational building blocks that comprise all our user interfaces.** These atoms include basic HTML elements like form labels, inputs, buttons, and others that can't be broken down any further without ceasing to be functional.



Atoms include basic HTML tags like inputs, labels, and buttons.

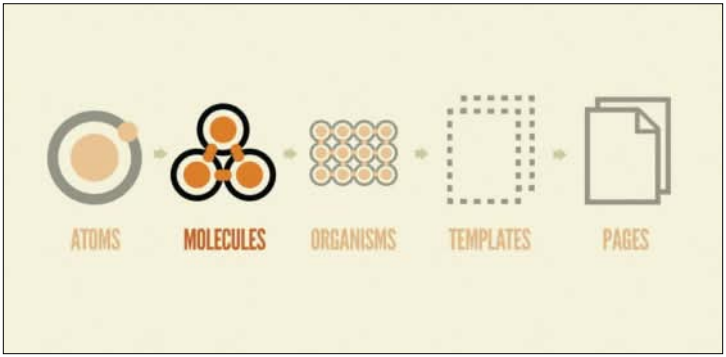
Each atom in the natural world has its own unique properties. A hydrogen atom contains one electron, while a helium atom contains two. These intrinsic chemical properties have profound effects on their application (for example, the Hindenburg explosion was so catastrophic because the airship was filled with extremely flammable hydrogen gas versus inert helium gas). In the same

42 https://developer.mozilla.org/en-US/docs/Web/HTML/Element

manner, each interface atom has its own unique properties, such as the dimensions of a hero image, or the font size of a primary heading. These innate properties influence how each atom should be applied to the broader user interface system.

In the context of a pattern library, atoms demonstrate all your base styles at a glance, which can be a helpful reference to keep coming back to as you develop and maintain your design system. But like atoms in the natural world, interface atoms don't exist in a vacuum and only really come to life with application.

Molecules



In chemistry, molecules are groups of atoms bonded together that take on distinct new properties. For instance, water molecules and hydrogen peroxide molecules have their own unique properties and behave quite differently, even though they're made up of the same atomic elements (hydrogen and oxygen).

In interfaces, **molecules are relatively simple groups of UI elements functioning together as a unit.** For example, a form label, search input, and button can join together to create a search form molecule.



A search form molecule is composed of a label atom, input atom, and button atom.

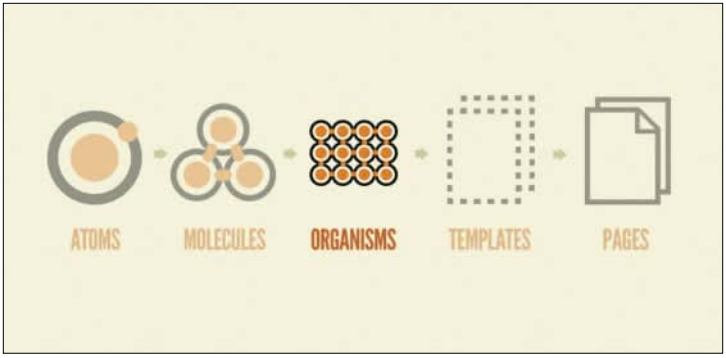
When combined, these abstract atoms suddenly have purpose. The label atom now defines the input atom. Clicking the button atom now submits the form. The result is a simple, portable, reusable component that can be dropped in anywhere search functionality is needed.

Now, assembling elements into simple functioning groups is something we've always done to construct user interfaces. But dedicating a stage in the atomic design methodology to these relatively simple components affords us a few key insights.

Creating simple components helps UI designers and developers adhere to the single responsibility principle, an age-old computer science precept that encourages a "do one thing and do it well" mentality. Burdening a single pattern with too much complexity makes software unwieldy. Therefore, creating simple UI molecules makes testing easier, encourages reusability, and promotes consistency throughout the interface.

Now we have simple, functional, reusable components that we can put into a broader context. Enter organisms!

Organisms



Organisms are relatively complex UI components composed of groups of molecules and/or atoms and/or other organisms. These organisms form distinct sections of an interface.

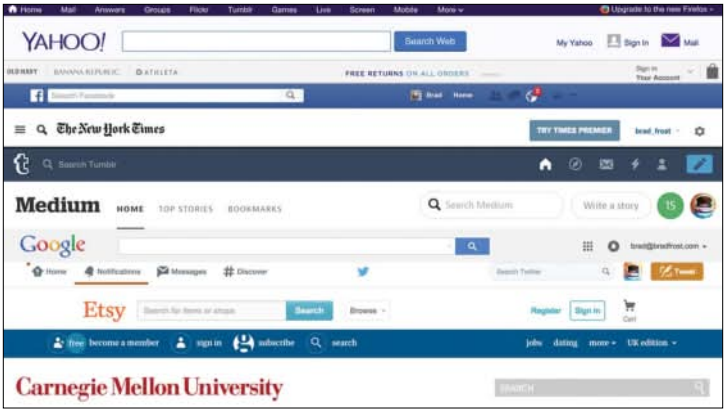
Let's revisit our search form molecule. A search form can often be found in the header of many web experiences, so let's put that search form molecule into the context of a header organism.



This header organism is composed of a search form molecule, logo atom, and primary navigation molecule.

The header forms a standalone section of an interface, even though it contains several smaller pieces of interface with their own unique properties and functionality.

Organisms can consist of similar or different molecule types. A header organism might consist of dissimilar elements such as a logo image, primary navigation list, and search form. We see these types of organisms on almost every website we visit.



Organisms like website headers consist of smaller molecules like primary navigation, search forms, utility navigation, and logos.

While some organisms might consist of different types of molecules, other organisms might consist of the same molecule repeated over and over again. For instance, visit a category page of almost any e-commerce website and you'll see a listing of products displayed in some form of grid.

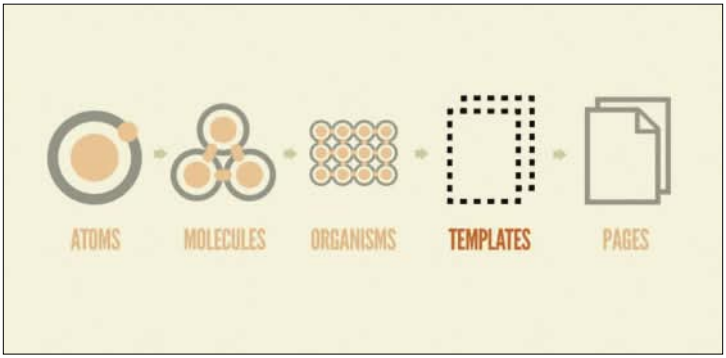
Building up from molecules to more elaborate organisms provides designers and developers with an important sense of context. Organisms demonstrate those smaller, simpler components in action and serve as distinct patterns that can be used again and again. The product grid organism can be employed anywhere a group of products needs to be displayed, from category listings to search results to related products.

Now that we have organisms defined in our design system, we can break our chemistry analogy and apply all these components to something that resembles a web page!



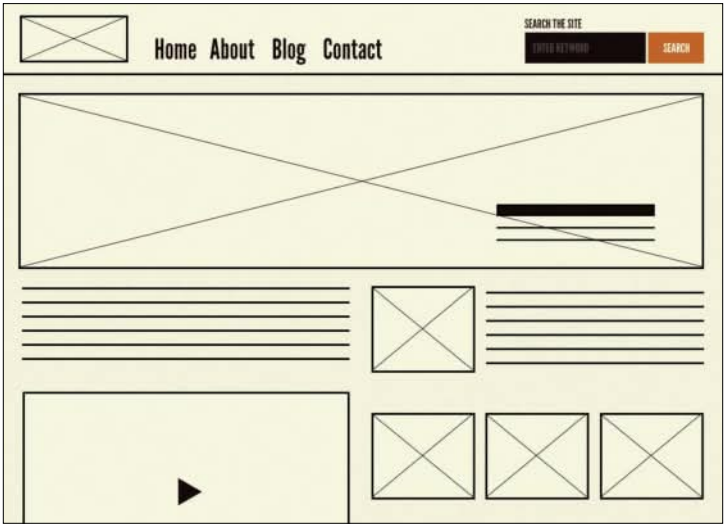
A product grid organism on Gap's e-commerce website consists of the same product item molecule repeated again and again.

Templates



Now, friends, it's time to say goodbye to our chemistry analogy. The language of atoms, molecules, and organisms carries with it a helpful hierarchy for us to deliberately construct the components of our design systems. But ultimately we must step into language that is more appropriate for our final output and makes more sense to our clients, bosses, and colleagues. Trying to carry the chemistry analogy too far might confuse your stakeholders and cause them to think you're a bit crazy. Trust me.

Templates are page-level objects that place components into a layout and articulate the design's underlying content structure. To build on our previous example, we can take the header organism and apply it to a homepage template.



The homepage template consists of organisms and molecules applied to a layout.

This homepage template displays all the necessary page components functioning together, which **provides context for these relatively abstract molecules and organisms**. When crafting an effective design system, it's critical to demonstrate how components look and function together in the context of a layout to

prove the parts add up to a well-functioning whole. We'll discuss this more in a bit.

Another important characteristic of templates is that they **focus on the page's underlying content structure** rather than the page's final content. Design systems must account for the dynamic nature of content, so it's very helpful to articulate important properties of components like image sizes and character lengths for headings and text passages.

Mark Boulton discusses the importance of defining the underlying content structure of a page:

> *You can create good experiences without knowing the content. What you can't do is create good experiences without knowing your content structure. What is your content made from, not what your content is.*
>
> – Mark Boulton

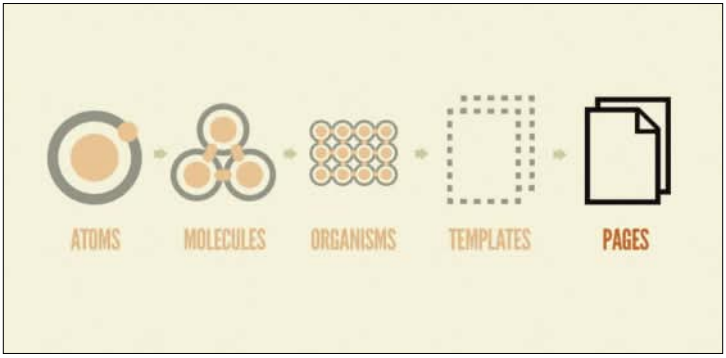
By defining a page's skeleton we're able to create a system that can account for a variety of dynamic content, all while providing needed guardrails for the types of content that populate certain design patterns. For example, the homepage template for Time Inc. shows a few key components in action while also demonstrating content structure regarding image sizes and character lengths:



Time Inc.'s homepage template demonstrates the content's underlying structure.

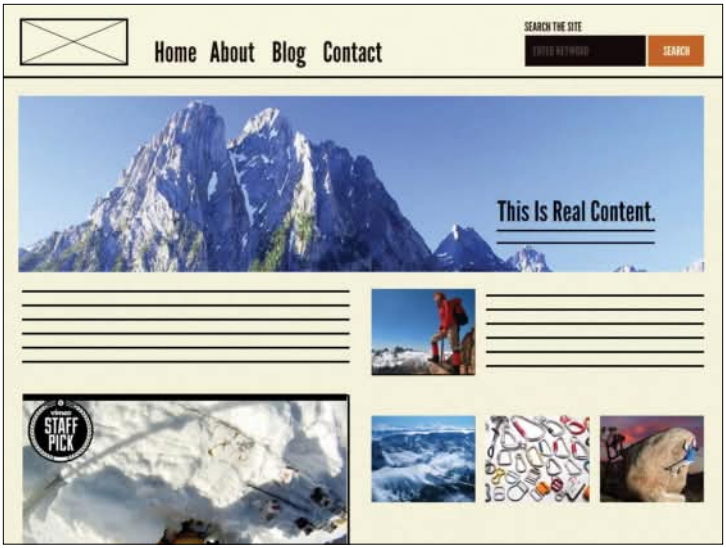
Now that we've established our pages' skeletal system, let's put some meat on them bones!

Pages



Pages are specific instances of templates that show what a UI looks like with real representative content in place. Building on our previous example, we can take the homepage template and pour representative text, images, and media into the template to show real content in action.

The page stage is the most concrete stage of atomic design, and it's important for some rather obvious reasons. After all, this is what users will see and interact with when they visit your experience. This is what your stakeholders will sign off. And this is where you see all those components coming together to form a beautiful and functional user interface. Exciting!



The page stage replaces placeholder content with real representative content to bring the design system to life.

In addition to demonstrating the final interface as your users will see it, **pages are essential for testing the effectiveness of the underlying design system**. It is at the page stage that we're able to take a look at how all those patterns hold up when real content is applied to the design system. Does everything look great and function as it should? If the answer is no, then we can loop back and modify our molecules, organisms, and templates to better address our content's needs.

When we pour real representative content into Time Inc.'s homepage template, we're able to see how all those underlying design patterns hold up.



At the page stage, we're able to see what Time Inc.'s homepage looks like with real representative content in place. With actual content in place, we're able to see if the UI components making up the page properly serve the content being poured into them.

We must create systems that establish reusable design patterns and also accurately reflect the reality of the content we're putting inside of those patterns.

Pages also provide a place to articulate variations in templates, which is crucial for establishing robust and reliant design systems. Here are just a few examples of template variations:

- A user has one item in their shopping cart and another user has ten items in their cart.
- A web app's dashboard typically shows recent activity, but that section is suppressed for first-time users.
- One article headline might be 40 characters long, while another article headline might be 340 characters long.
- Users with administrative privileges might see additional buttons and options on their dashboard compared to users who aren't admins.

In all of these examples, the underlying templates are the same, but the user interfaces change to reflect the dynamic nature of the content. These variations directly influence how the underlying molecules, organisms, and templates are constructed. Therefore, creating pages that account for these variations helps us create more resilient design systems.

So that's atomic design! These five distinct stages concurrently work together to produce effective user interface design systems. To sum up atomic design in a nutshell:

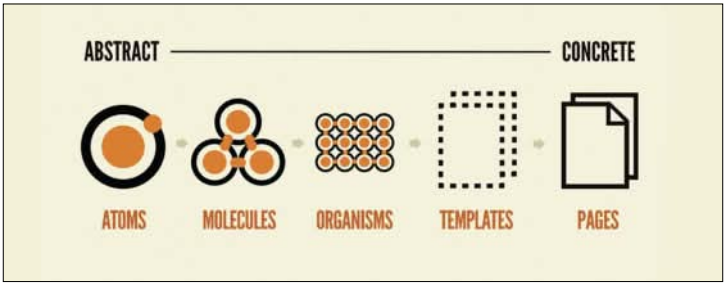
- **Atoms** are UI elements that can't be broken down any further and serve as the elemental building blocks of an interface.
- **Molecules** are collections of atoms that form relatively simple UI components.
- **Organisms** are relatively complex components that form discrete sections of an interface.
- **Templates** place components within a layout and demonstrate the design's underlying content structure.
- **Pages** apply real content to templates and articulate variations to demonstrate the final UI and test the resilience of the design system.

Advantages of atomic design

So why go through all this rigamarole? What's atomic design good for? These are valid questions, considering we've been building user interfaces for a long time now without having an explicit five-stage methodology in place. But atomic design provides us with a few key insights that help us create more effective, deliberate UI design systems.

The part and the whole

One of the biggest advantages atomic design provides is the ability to quickly shift between abstract and concrete. We can simultaneously see our interfaces broken down to their atomic elements and also see how those elements combine together to form our final experiences.



Atomic design allows designers to traverse between abstract and concrete.

In his book *The Shape of Design*, Frank Chimero beautifully articulates the power this traversal provides:

> *The painter, when at a distance from the easel, can assess and analyze the whole of the work from this vantage. He scrutinizes and listens, chooses the next stroke to make, then approaches the canvas to do it. Then, he steps back again to see what he's done in relation to the whole. It is a dance of switching contexts, a pitter-patter pacing across the studio floor that produces a tight feedback loop between mark-making and mark-assessing.*
>
> – Frank Chimero

Atomic design lets us dance between contexts like the painter Frank so eloquently describes. The atoms, molecules, and organisms that comprise our interfaces do not live in a vacuum. And our interfaces' templates and pages are indeed composed of smaller parts. The parts of our designs influence the whole, and the whole influences the parts. The two are intertwined, and atomic design embraces this fact.

When designers and developers are crafting a particular component, we are like the painter at the canvas creating detailed strokes. When we are viewing those components in the context of a layout with real representative content in place, we are like the painter several feet back from the canvas assessing how their detailed strokes affect the whole composition. It's necessary to zero in on one particular component to ensure it is functional, usable, and beautiful. But it's also necessary to ensure that component is functional, usable, and beautiful *in the context of the final UI.*

Atomic design provides us a structure to navigate between the parts and the whole of our UIs, which is why it's crucial to reiterate that **atomic design is not a linear process**. It would be foolish to design buttons and other elements in isolation, then cross our fingers and hope everything comes together to form a cohesive whole. So don't interpret the five stages of atomic design as "Step 1: atoms; Step 2: molecules; Step 3: organisms; Step 4: templates; Step 5: pages." Instead, **think of the stages of atomic design as a mental model that allows us to concurrently create final UIs and their underlying design systems.**

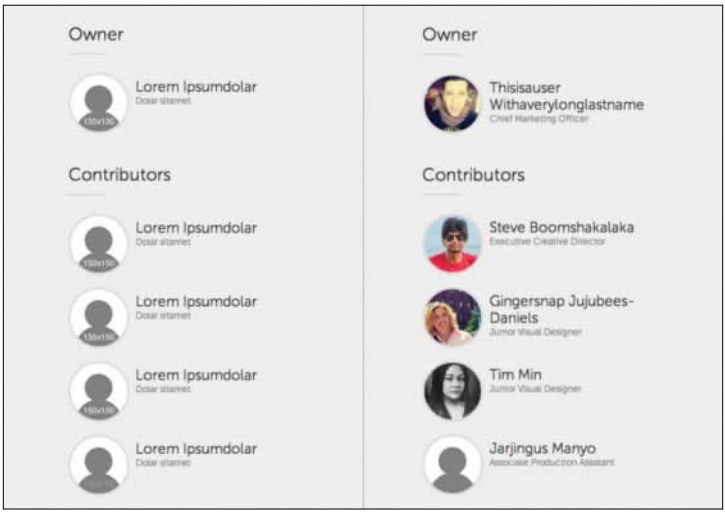
Clean separation between structure and content

Discussing *design* and *content* is a bit like discussing the *chicken* and the *egg*. Mark Boulton explains:

> *Content needs to be structured and structuring alters your content, designing alters content. It's not 'content then design', or 'content or design'. It's 'content and design'.*
>
> – Mark Boulton

A well-crafted design system caters to the content that lives inside it, and well-crafted content is aware of how it's presented in the context of a UI. The interface patterns we establish must accurately reflect the nature of the text, images, and other content that live inside them. Similarly, our content should be aware of the manner in which it will be presented. The close relationship between content and design requires us to consider both as we construct our UIs.

Atomic design gives us a language for discussing the structure of our UI patterns and also the content that goes inside those patterns. While there is a clean separation between the content structure skeleton (templates) and the final content (pages), atomic design recognizes the two very much influence each other. For instance, take the following example:



On the left we see the UI's content skeleton, which consists of the same *person block molecule* repeated again and again. On the right we see what happens when we populate each instance of the person block molecule with representative content. Visualizing the content skeleton and the representative final content allows us to create patterns that accurately reflect the content that lives inside them. If a person's name were to wrap onto five lines within the pattern, we would need to address that broken behavior at a more atomic level.

The content we pour into our UIs at the page stage will influence the characteristics and parameters of the underlying design patterns.

What's in a name?

Throughout this book I've mentioned that modular design and development is nothing new. So why are we introducing terms like *atoms*, *molecules*, and *organisms* when we can just stick with established terms like *modules*, *components*, *elements*, *sections*, and *regions*?

For as long as I've been talking about atomic design, I've had people proffer alternate names for the stages of the methodology. Person One would suggest, "Why not just name them elements, modules, and components?" while Person Two would suggest, "Why not just name them base, components, and modules?" The issue with terms like components and modules is that a sense of hierarchy can't be deduced from the names alone. **Atoms, molecules, and organisms imply a hierarchy** that anyone with a basic knowledge of chemistry can hopefully wrap their head around.

That being said, naming things is hard and imperfect. The names I've chosen for the stages of atomic design have worked really well for me and the teams I've worked with as we create UI design systems. But maybe they don't work for you and your organization. That's more than OK. Here's one perspective from the design team at GE:

> *As we showed our initial design system concepts that used the Atomic Design taxonomy to our colleagues, we were met with the some confused looks. [...] The evidence was clear, for this to be successful within our organization we had to make the taxonomy more approachable.*
>
> – Jeff Crossman, GE Design

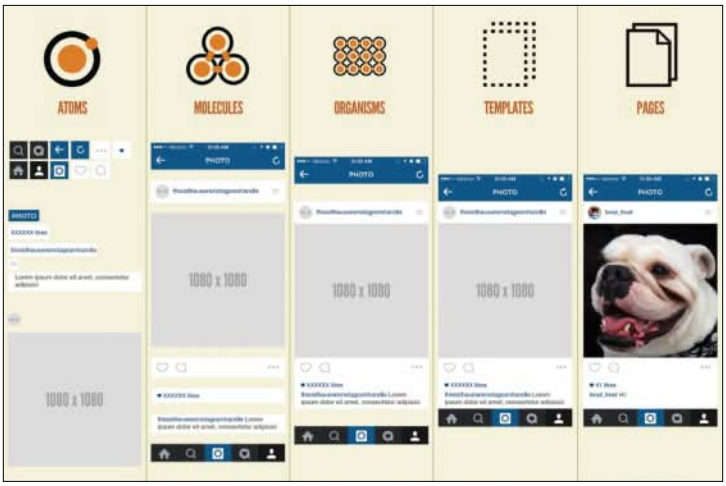
The taxonomy the team landed on were "Principles", "Basics", "Components", "Templates", "Features", and "Applications". Do these labels make sense to you? It doesn't matter. By establishing a taxonomy that made sense for their organization, everyone was able to get on board with atomic design principles and do effective work together.

"Atomic design" as a buzzword encapsulates the concepts of modular design and development, which becomes a useful shorthand for convincing stakeholders and talking with colleagues. But **atomic design is not rigid dogma**. Ultimately, whatever taxonomy you choose to work with should help you and your organization communicate more effectively in order to craft an amazing UI design system.

Atomic design is for user interfaces

Atomic design is a concept born of the web. After all, your lowly author is a web designer, which is mainly the reason this book primarily focuses on web-based interfaces. But it's important to understand that **atomic design applies to all user interfaces, not just web-based ones.**

You can apply the atomic design methodology to the user interface of any software: Microsoft Word, Keynote, Photoshop, your bank's ATM, whatever. To demonstrate, let's apply atomic design to the native mobile app Instagram.



Atomic design applied to the native mobile app Instagram.

Let's walk through this atomized Instagram interface:

- **Atoms:** This screen of Instagram's UI consists of a handful of icons, some text-level elements, and two image types: the primary image and the user's avatar image.
- **Molecules:** Several icons form simple utilitarian components like the bottom navigation bar and the photo actions bar where users can like or comment on a photo. Also, simple combinations of text and/or images form relatively simple components.
- **Organisms:** Here we can see the photo organism take shape, which consists of the user's information, time stamp, the photo itself, actions around that photo, and information about the photo including like count and caption. This organism becomes the cornerstone of the entire Instagram experience as it is stacked repeatedly in a never-ending stream of user-generated photos.
- **Templates:** We get to see our components come together in the context of a layout. Also, it's here where we see the exposed content skeleton of the Instagram experience, highlighting dynamic content such as the user's handle, avatar, photo, like count, and caption.
- **Pages:** And finally we see the final product, complete with real content poured into it, which helps ensure the underlying design system comes together to form a beautiful and functional UI.

I show this non-web example because atomic design tends to get misinterpreted as an approach to web-specific technologies like CSS and JavaScript. Let me be clear about this: **atomic design has nothing to do with web-specific subjects like CSS or JavaScript architecture.** In chapter 1 we discussed the trend toward modularity in all aspects of design and development, which includes CSS and JavaScript. These are fantastic trends in CSS and JavaScript, but atomic design deals with crafting user interface design systems irrespective of the technology used to create them.

Atomic design in theory and in practice

This chapter introduced the atomic design methodology and demonstrated how atoms, molecules, organisms, templates, and pages all work together to craft thoughtful, deliberate interface design systems. Atomic design allows us to see our UIs broken down to their atomic elements, and also allows us to simultaneously step through how those elements join together to form our final UIs. We learned about the tight bond between content and design, and how atomic design allows us to craft design systems that are tailored to the content that lives inside them. And finally we learned how the language of atomic design gives us a helpful shorthand for discussing modularity with our colleagues, and provides a much needed sense of hierarchy in our design systems.

Atomic design is a helpful design and development methodology, but essentially it's merely a mental model for constructing a UI. By now you may be wondering *how* you make atomic design happen. Well, fear not, dear reader, because the rest of the book focuses on tools and processes to make your atomic design dreams come true.